Elements of AIML Lab
Experiment 04 (After-submission Reference): Pandas Import/Export and Summaries

Tofik Ali

March 1, 2026

Contents

1 How to use this reference **1**

2 What this experiment is really testing **1**

3 Worked example (self-contained) **2**
3.1 Step 0: create a small example CSV (only for demonstration) 2
3.2 Step 1: import the dataset 2
3.3 Step 2: basic inspection (what each command tells you) 2
3.4 Step 3: missing values and duplicates 2
3.5 Step 4: descriptive summaries 2
3.6 Step 5: a small clean step + export 3

4 What to write in the report (example phrasing) **3**

5 Troubleshooting (common issues and fixes) **3**

1 How to use this reference

This reference is meant to be shared **after you submit** Experiment 04. It provides a complete, readable walkthrough of the **import** → **inspect** → **clean** → **export** cycle.

2 What this experiment is really testing

Before we build ML models, we must be able to answer:

- What are the columns and data types?
- How many rows/columns do we have?
- Are there missing values, duplicates, impossible values?
- Can we export a processed dataset for later experiments?

3 Worked example (self-contained)

3.1 Step 0: create a small example CSV (only for demonstration)

If you already used a real dataset, skip to Step 1. The code below creates an example file.

```
import pandas as pd

df_demo = pd.DataFrame({
    "age": [21, 22, None, 24, 24],
    "city": ["Dehradun", "Delhi", "Delhi", None, "Mumbai"],
    "marks": [78, 85, 66, 92, 92],
})
df_demo.to_csv("demo.csv", index=False)
```

3.2 Step 1: import the dataset

```
import pandas as pd

df = pd.read_csv("demo.csv") # replace with your dataset path
```

If your CSV uses a different separator, try `sep=";"` (or check the file).

3.3 Step 2: basic inspection (what each command tells you)

```
print("Shape:", df.shape) # (rows, columns)
print(df.head()) # preview
print(df.info()) # dtypes + non-null counts
```

How to interpret:

- `shape`: helps you confirm import success (not empty, not truncated).
- `head`: checks if columns look correct (no shifted columns due to commas).
- `info`: shows types; if a numeric column appears as `object`, it may contain strings like "–".

3.4 Step 3: missing values and duplicates

```
missing = df.isna().sum().sort_values(ascending=False)
print("Missing per column:\n", missing)
print("Duplicates:", df.duplicated().sum())
```

Interpretation: columns with high missingness may require imputation or removal in later experiments.

3.5 Step 4: descriptive summaries

```
print(df.describe()) # numeric columns
print(df.describe(include="object")) # categorical columns
```

Look for:

- suspicious min/max (data entry errors/outliers),
- impossible values (negative age),
- rare categories (very low counts).

3.6 Step 5: a small clean step + export

One realistic mini-clean is: drop duplicates, fill a missing category with `"Unknown"`, and export.

```
df2 = df.drop_duplicates()
df2["city"] = df2["city"].fillna("Unknown")
df2.to_csv("processed.csv", index=False)
```

Cross-check: Open `processed.csv` and confirm it contains the expected rows/columns.

4 What to write in the report (example phrasing)

- "The dataset has 1200 rows and 14 columns. The column `Age` has 23 missing values (1.9%)."
- "`info()` shows `Income` is `object`; we found currency symbols and removed them (converted to float)."
- "We exported `processed.csv` after dropping duplicates and filling missing city values with `Unknown`."

5 Troubleshooting (common issues and fixes)

- **Unicode errors:** try `pd.read_csv(..., encoding="utf-8")` or `"latin-1"`.
- **Wrong separator:** use `sep=";"` or `sep="
t"` for TSV.
- **Numeric column read as text:** use `pd.to_numeric(..., errors="coerce")`.
- **Dates read as strings:** use `parse_dates=[...]` or `pd.to_datetime`.

Sources: Provost and Fawcett, *Data Science for Business.*;McKinney, *Python for Data Analysis.*;WEKA Documentation (Waikato Environment for Knowledge Analysis), accessed 2026-02-28.;Matplotlib and Seaborn documentation, accessed 2026-02-28.